

Multi-Source Candidate Data Transformer — Design (Stage 1)

Turn messy, conflicting, multi-source candidate inputs into **one canonical profile** with provenance and confidence on every value. Principle: *wrong-but-confident is worse than honestly-empty* — unknowns become null, never invented.

Pipeline

ingest → extract (per-source adapters) → normalize → merge/dedupe → score confidence → canonical record → project (runtime config) → validate → emit

extract: each adapter emits raw claims (*path, value, source, method*) and **never throws** — a bad source returns [] + a warning. **normalize**: pure functions canonicalize; a value that won't normalize is **dropped**. **merge**: claims → one CandidateProfile (single source of truth).

Canonical schema & normalized formats

Fields: candidate_id, full_name, emails[], phones[], location{city,region,country}, links{linkedin,github,portfolio,other[]}, headline, years_experience,

skills[{name,confidence,sources[],verified_in_code}], experience[{company,title,start,end,summary}], education[{institution,degree,field,end_year}], provenance[{field,source,method}], overall_confidence.

Phones → E.164; dates → YYYY-MM (a bare year is dropped, never invented as -01); country → ISO-3166 alpha-2 (exact or flagged-fuzzy); skills → canonical via an alias dictionary (js→JavaScript), unknowns kept & acronyms preserved; emails lowercased/validated.

Sources (≥1 structured + ≥1 unstructured)

Structured: **Recruiter CSV, ATS JSON** (foreign field names → an **explicit remap dict**; unmapped keys logged, never guessed). Unstructured: **GitHub** (recorded fixture by default, --live for the real API), **Resume PDF/DOCX, Recruiter notes**.

Merge & conflict resolution

Identity / candidate_id — **3 tiers**: (a) hash of best email; (b) name + matching phone; (c) name + source-file identity. **Zero matchable identifiers** → **no cross-source dedupe** (kept standalone). **Scalars**: highest source trust wins (tie-broken by completeness, then stable order). **Lists** unioned & de-duped.

Corroboration is case-insensitive. **Per skill verified_in_code**: each skill is {name, confidence, sources[], verified_in_code}; the flag is true iff github is among its sources — a skill claimed in a resume but absent from the candidate's public code carries this flag rather than being silently trusted as equally strong, and it does not alter confidence or add a scoring path.

Confidence (concrete math)

Source	ATS	CSV	Resume	GitHub	Notes
Base trust	0.90	0.85	0.60	0.50	0.35

```
per_field = base_trust(winner) + 0.05 * min(corroborations, 3) # ≤ +0.15
per_field *= 0.9 if value is fuzzy/regex-derived (resume regex, fuzzy country)
per_field = clamp(per_field, 0.05, 0.99) # never 1.0
overall = importance-weighted mean (identity fields weighted higher)
```

Worked examples (all reproducible from the shipped samples): Jane full_name = ATS "Jane McDonald" (0.90) + CSV+GitHub+resume corroboration (+0.15) → clamp **0.99**; Liang location.country "The Netherlands"→NL is fuzzy, from ATS (0.90) + GitHub (+0.05 = 0.95) × 0.9 = **0.855**; Jane PostgreSQL skill, resume-only via regex (fuzzy) → 0.60×0.9 = **0.54**. (*Hypothetical, not in samples: a notes-only skill would floor at base 0.35.*)

Runtime config (projection + validation)

Config selects/renames fields (from DSL: emails[0], skills[].name), sets per-field normalization, toggles provenance/confidence, and chooses a missing-value policy driving **three schema shapes**: **omit** → not required; **null** → required + nullable; **error** → required + non-null (missing fails loudly). Canonical record and projection are strictly separated — default and any custom shape come from **one engine, no code changes**.

Edge cases handled

(1) Conflicting name casing → trust winner kept, others still corroborate. (2) Malformed phone / bare-year date → dropped, stays null. (3) Malformed JSON / empty CSV → skipped with a warning; other sources still produce a profile. (4) Same skill, many spellings → canonicalized & merged, sources combined. (5) No email & no phone match → kept standalone; dedupe not attempted.

Out of scope (under time pressure)

LinkedIn scraping (no public API); ML/NLP resume parsing (deterministic regex/section heuristics, flagged low-confidence); fuzzy cross-candidate identity beyond the 3-tier chain; a UI (a clean CLI is provided).